

Draw It or Lose It

CS 230 Project Software Design

Version 3.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	09/20/26	Devon Parker	Initial draft of the software design. This includes executive summary, constraints, domain model analysis, platform evaluation, and final recommendations.

Version	Date	Author	Comments
2.0	10/04/26	Devon Parker	Second draft with an updated Evaluation table.

Version	Date	Author	Comments
3.0	10/25/26	Devon Parker	Final draft with Recommendations section complete.

Executive Summary

The Gaming Room is transitioning their current Android-only game, Draw It or Lose It, into a web-based distributed application to reach a broader audience across multiple platforms. This document outlines the software design, platform evaluations, and architectural recommendations necessary to accomplish this goal. The proposed solution uses a Java back end implementing Object-Oriented principles and Singleton or Iterator design patterns to manage memory and ensure unique naming conventions. To proceed, the client must review and approve the recommended Client-Server architecture and the Linux-based hosting environment detailed in this proposal.

Requirements

- The game must support one or more teams, each with multiple players.
- Game, team, and player names must be uniquely identifiable to prevent duplication.
- Only one instance of the game engine can exist in memory at any given time.
- The game must enforce strict timers: one-minute rounds, drawings complete at 30 seconds, and 15 seconds for opposing teams to guess if time expires.
- The application must be web-based to support cross-platform compatibility.

Design Constraints

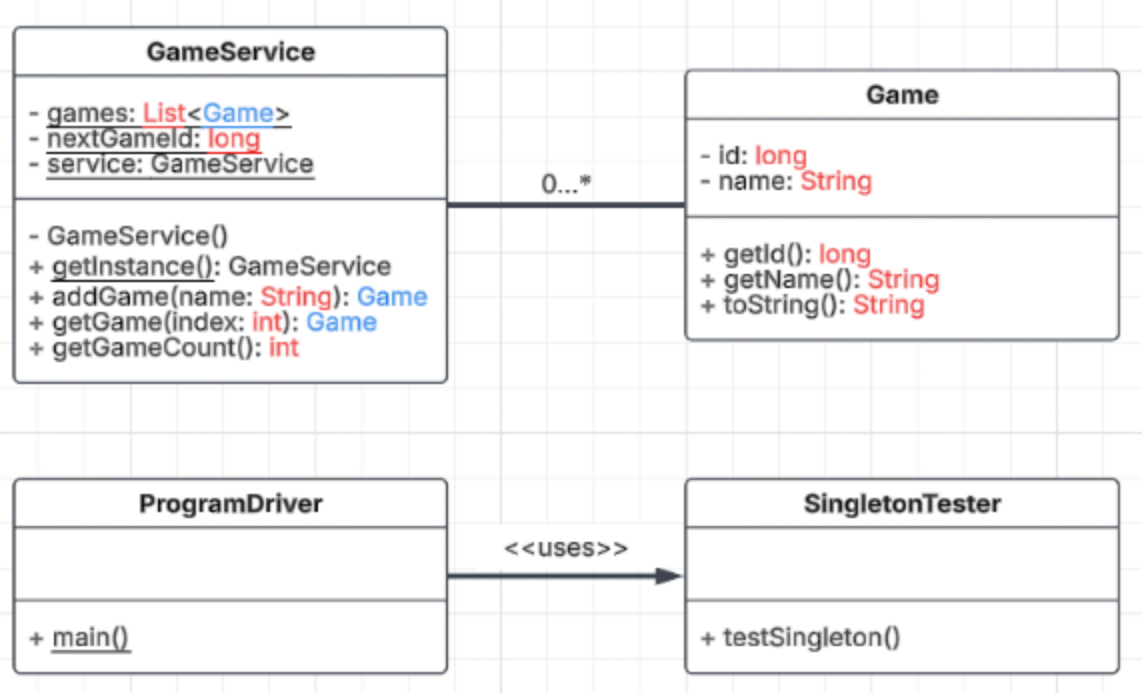
Developing a web-based distributed environment introduces constraints around network latency, state synchronization, and memory management. Because the game relies on precise timers, the architecture must minimize lag between the server and clients. Furthermore, the requirement that only one instance of the game exists in memory necessitates the use of a Singleton pattern on the back end, which restricts how the application scales horizontally and requires careful handling of concurrent user requests.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram uses Object-Oriented inheritance and composition to structure the application efficiently. The Entity base class holds the common attributes (id and name) and methods (getId(), getName()) for the system. The Game, Team, and Player classes all inherit from Entity, eliminating redundant code and promoting reusability. The diagram also has a nested one-to many relationship structure: the GameService maintains a list of Game objects, each Game contains a list of Team objects, and each Team contains a list of Player objects. The GameService is designed as a Singleton so only one instance manages the active games in memory.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac servers are rarely used for enterprise web hosting and lack the scalable deployment methods required for thousands of players. Deploying on macOS would require expensive, proprietary Apple hardware, and the server software is largely deprecated for this type of distributed hosting, making it a poor choice for the client.	Linux is the industry standard for hosting Java-based web applications and offers highly scalable deployment methods. A major advantage for The Gaming Room is that Linux distributions are open-source, meaning there are no licensing costs for the server operating system, keeping overhead extremely low.	Windows offers a robust server deployment method through IIS and is highly reliable. However, the client would face significant licensing costs for the Windows Server operating system, which scales up depending on the hardware and number of server instances required.	Mobile platforms lack the processing power, constant network connectivity, and architectural design required to act as a server for a distributed web application. There are no server-based deployment methods or server OS licensing costs to consider, as this is an impossible hosting environment.

Client Side	Since the application is web-based, Mac users will access it via standard browsers like Safari or Chrome. The development process requires standard web expertise to ensure compatibility. This keeps costs and time low, as developers do not need specialized macOS programming knowledge.	Linux clients will access the game through web browsers. The development process relies on the same standard web interface used for Mac and Windows, ensuring the application is compatible across all platforms without requiring additional Linux-specific expertise, saving both time and development costs.	Windows represents a massive user base that will access the game via browsers. Delivering a responsive HTML interface requires standard front-end development expertise. Because the same web codebase serves Windows, Mac, and Linux, the development time and costs are highly optimized.	Delivering the web interface to iOS and Android requires mobile-first design expertise. Developers must invest additional time to ensure the responsive HTML UI scales correctly on touchscreens and smaller viewports. While this requires more effort than desktop sizing, using a single web application still costs less than building separate native iOS and Android apps.
Development Tools	Developers will use Java for the back end and HTML/JS for the front end, utilizing cross-platform IDEs like IntelliJ IDEA, Eclipse, or Visual Studio Code. Because these tools handle web and Java development natively, a single development team can build the application. Most of these tools offer free or open-source community editions, eliminating licensing costs.	The programming languages and tools remain identical on Linux. The technical requirements allow the exact same development team to work seamlessly across environments without needing platform-specific specialists. The tools are open-source, resulting in zero licensing costs.	Windows developers will utilize the identical stack and IDEs. The technical impact on the team is minimal since a single full-stack web team can handle the entire project across all desktop environments. IDE community editions are free, though enterprise versions would incur optional licensing costs.	The same web languages are used, meaning no native Swift or Kotlin tools are required. Developers will use browser-based mobile emulators within their desktop IDEs to test the software. This means multiple specialized mobile development teams are not needed. The core web team can handle mobile deployment. There are no extra licensing costs for these browser-testing tools.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: I recommend using a Linux cloud server to host the back end and run the web application for players. It is reliable, keeps hosting costs low since there are no licensing fees, and makes running the Java application simple.

2. **Operating Systems Architectures**: The game will use a client-server setup where the main Linux server handles the game rules, timers, and score tracking. Players can then easily join in using a standard web browser on almost any device.

3. **Storage Management**: A relational database system like PostgreSQL or MySQL is the best fit for storing player accounts, scores, and drawing words. It keeps all the data organized and safe while matching up naturally with how the game code is structured.

4. **Memory Management**: The server will run the game inside the Java Virtual Machine, which automatically cleans up old memory when game sessions end. We also use the Singleton design pattern so the server only ever keeps one main game engine running in memory at a time.

5. **Distributed Systems and Networks**: To keep players on different devices synced up in real time, the system will use WebSockets over the network so drawing updates and guesses happen instantly. If someone loses their connection during a game, the server will handle the dropout smoothly so it does not freeze up for everyone else.

6. **Security**: All connection traffic between players and the server will be encrypted using HTTPS and TLS, with token logins to keep accounts safe. The Linux server will also use built-in firewalls and back-end checks to block common threats like SQL injection.